SYSTEM DESIGN DIAGRAMS

for

ShadePaths

Version 1.0

Prepared by Group A:
Raimundo Robredo Amaro
Ken Sunamoto
Mattu Yuvaraj Singh
Pyae Su Khaing
Suyama Riku
Yamaguchi Haruho

Made for PBL3
Ritsumeikan University

Last Modified on: 17/06/2026

Table of Contents

List of Figures

1. Introduction

This section of the System Design Diagram (SDD) document provides the purpose and intended audience for the SDD document.

1.1. Purpose

This SDD describes in detail the components, architecture and the design decisions for ShadePaths with the purpose of supporting the development and future evaluation of the application.

1.2. Intended audience

This document is intended for the ShadePaths development team, the PBL course instructors who are in charge of reviewing it, and any future teams who may want to expand on the application.

2. Design Viewpoints

This section describes the design of ShadePaths using six viewpoints chosen to collectively address the context, structure, logic, interfaces, behavior, and core runtime of the system. These viewpoints were selected because ShadePaths has a non-trivial routing algorithm that must be isolated server-side, multiple external API dependencies, a clear user-facing interface with distinct application states, and a real-time navigation runtime, all of which require accurate structural and behavioral descriptions to support implementation. Each viewpoint entry includes the concern it addresses, the notation used, the view itself, and a description of what the view contributes beyond what the SRS already specifies.

2.1. Class Diagram

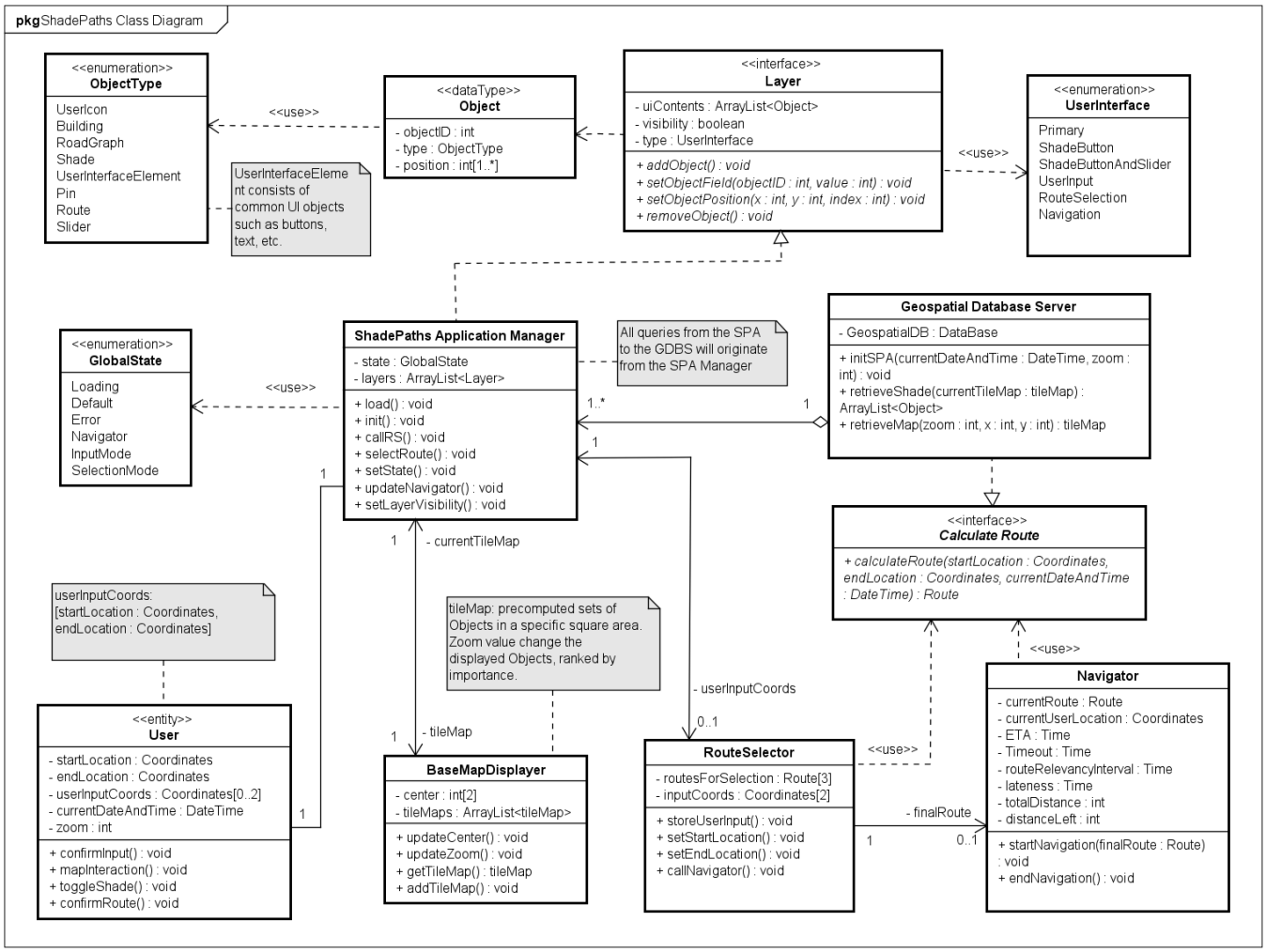

Figure 1: Class Diagram

Description

This class diagram shows the structure of the ShadePaths software system. The ShadePaths Application Manager (SPA Manager) contains all the details from the user and the control system of the SPA. SPA Manager contains the state of the Application and the user interface layers for display. The layer types are shown in the class UserInterface. The SPA Manager also contains the functionalities of the class User, where it contains the attributes and operations of individual users. Geospatial

Database Server provides required data through the SPA Manager.

Each layer has an ArrayList of Objects. An Object is a custom datatype where each object contributes to the creation of the user interface. The types of objects are shown in the class ObjectType. Detailed information stored in the GDBS are not stored in the client device, and calculations of routes and shadows are queried from the GDBS. Shade data are precomputed and already stored in the GDB, while the route calculations are computed when required.

The BaseMapDisplayer class contains the base map attributes and operations that are related to tileMaps. The tileMap acts as the base, where other layers are built on top of the base. RouteSelector and Navigator both contain functionalities for their subsystem counterparts. The Shade Overlay subsystem is implemented in the SPA Manager class so it doesn't require its own class.

Rationale

The SRS document defines subsystem behavior through stimulus/response tables but doesn't specify the internal structure of those subsystems, their attributes, which classes own which operations, or how state is shared across them. Specifically, the SRS never defines what GlobalState contains, who is responsible for calling callRS() vs updateNavigator(), or how the User entity maps to class-level attributes like currentDateAndTime and zoom. The class diagram provides the single authoritative reference for these structural decisions, which are required before any module can be written.

2.2. Use Case Diagram

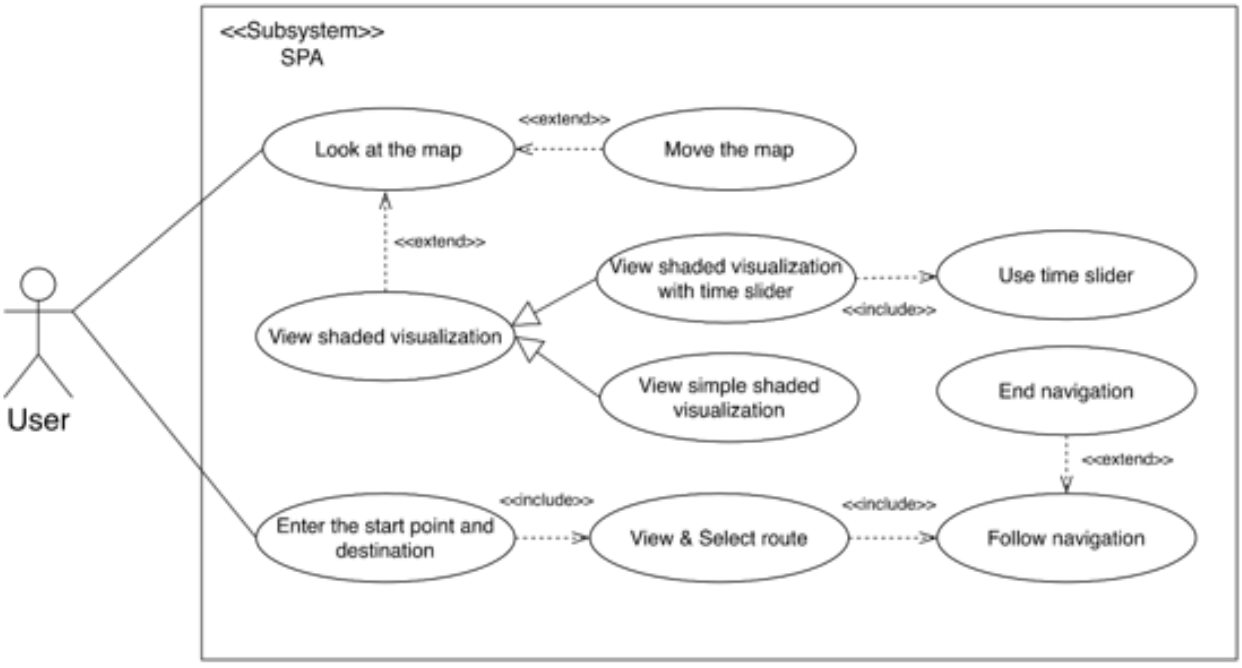

Figure 2: Use Case Diagram

Description

This use case diagram describes the core functions which are included in ShadePaths and the ways that a pedestrian user interacts with the system.The shade visualization is split into two specific use cases, one is a simple shaded visualization and the other is with a time slider feature so that the user can see shaded areas either for the current time or the shade shifts over the time in a day. View shaded visualization uses ≪extend≫ toward Look at the map because viewing shade is optional and only makes sense when the map is already visible. Move the map also uses ≪extend≫ toward Look at the map for the same reason since the user cannot move something they are not already looking at.

Rationale

This diagram is used to give a clear overview of the system before starting the phase of implementation. The functional requirements stated in the SRS document, it does not clearly show how users will be interacting with the system or how the different functions are going to relate to each other. Therefore, the use case diagram helps us to define the boundary of the system, the primary actor, and the main relationships between the necessary and optional functions. Due to those, it was included and created to ensure that all major user interactions are considered and agreed before the implementation stage.

2.3. Component Diagram

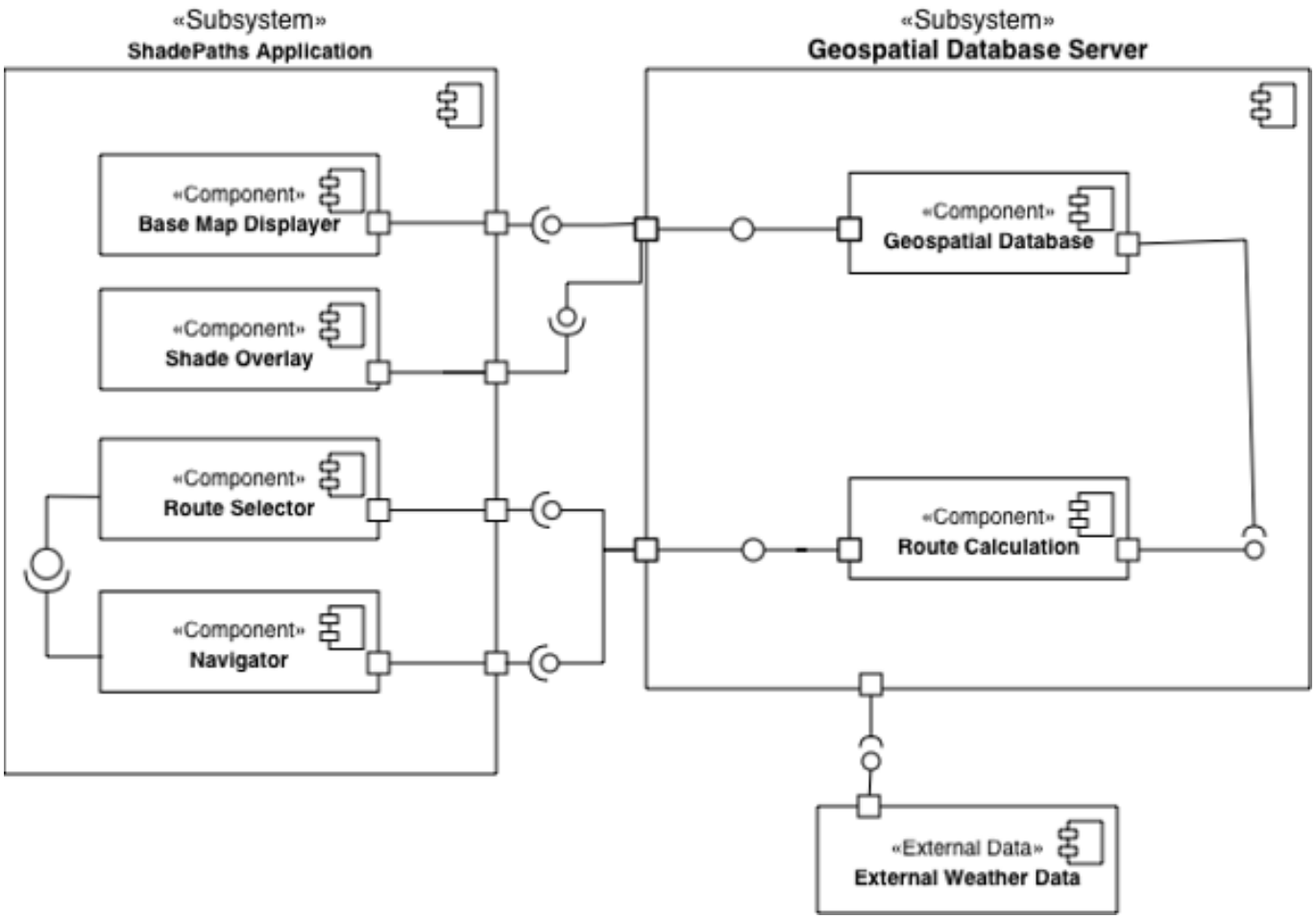

Figure 3: Component Diagram

Description

This component diagram shows how the main parts of the ShadePaths system are connected to each other. The two main subsystems in this system are the ShadePaths Application and the Geospatial Database Server. Base Map Displayer, Shade Overlay, Route Selector, and Navigator are the four components included in the application side.

On the server side, the Geospatial Database manages map-related data. The Route Calculation component uses this data to calculate possible walking routes. The diagram also includes External Weather Data as an external data source. The ports and socket connections show which components require data or services and which components provide them. Overall, this diagram explains how the application components communicate with the server components to provide shaded route navigation.

Rationale

This component diagram was created to show the internal structure of the ShadePaths system more clearly before implementation. The main purpose is to show how the system works behind the screen. Therefore, this component diagram helps understanding which components are necessary, what each component is responsible for, and how data moves between the application and the server.

It is especially useful because ShadePaths depend on several different types of data. Without this diagram, it would be difficult to explain which part handles the map display, which part calculates routes, and which part provides geospatial information. Also, by separating the application side and the server side, we can see that the user interface components do not directly do all the processing. Instead, they request data or services from the server components. This makes the system easier to understand and to develop as separate parts. For these reasons, this diagram was included to organize the system design.

2.4. State Machine Diagram

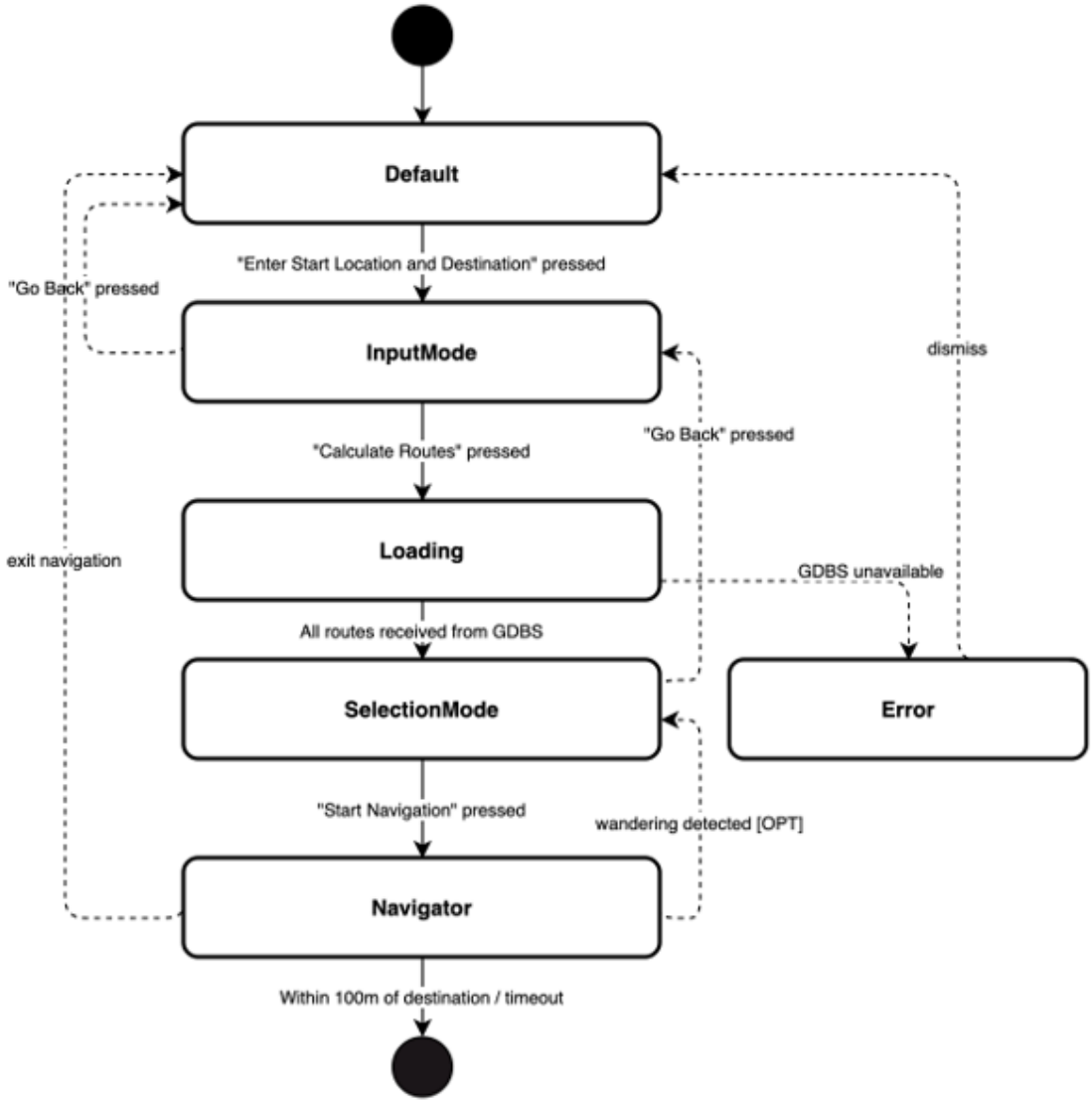

Figure 4: State Machine Diagram

Description

Every state depicted in the diagram has exactly one corresponding value. As each transition signifies a call to the setState() method, the diagram is the definition of what exactly the method needs to do.

The RouteSelector sub-states share a similar goal to gather user input and fetch routes from the GDBS. "Go Back" and "Exit Navigation" transitions allow the user to step back or leave the current state at any point before or during navigation.

The dismiss transition from Error returns to Default, discarding userInputCoords and requiring the user to reenter locations from scratch, ensuring the next request starts clean. Cancel navigation from Navigator also returns directly to Default, simplifying Navigator's cleanup logic by not needing to restore RouteSelector's prior state. The Loading state has no cancel path, so GDBS timeout must be handled server side before the client triggers the Error transition. Finally, the wandering detected transition returns to Loading rather than applying a silent reroute, bringing the user back to SelectionMode with updated options and keeping them in control of any route change.

2.5. Sequence Diagram

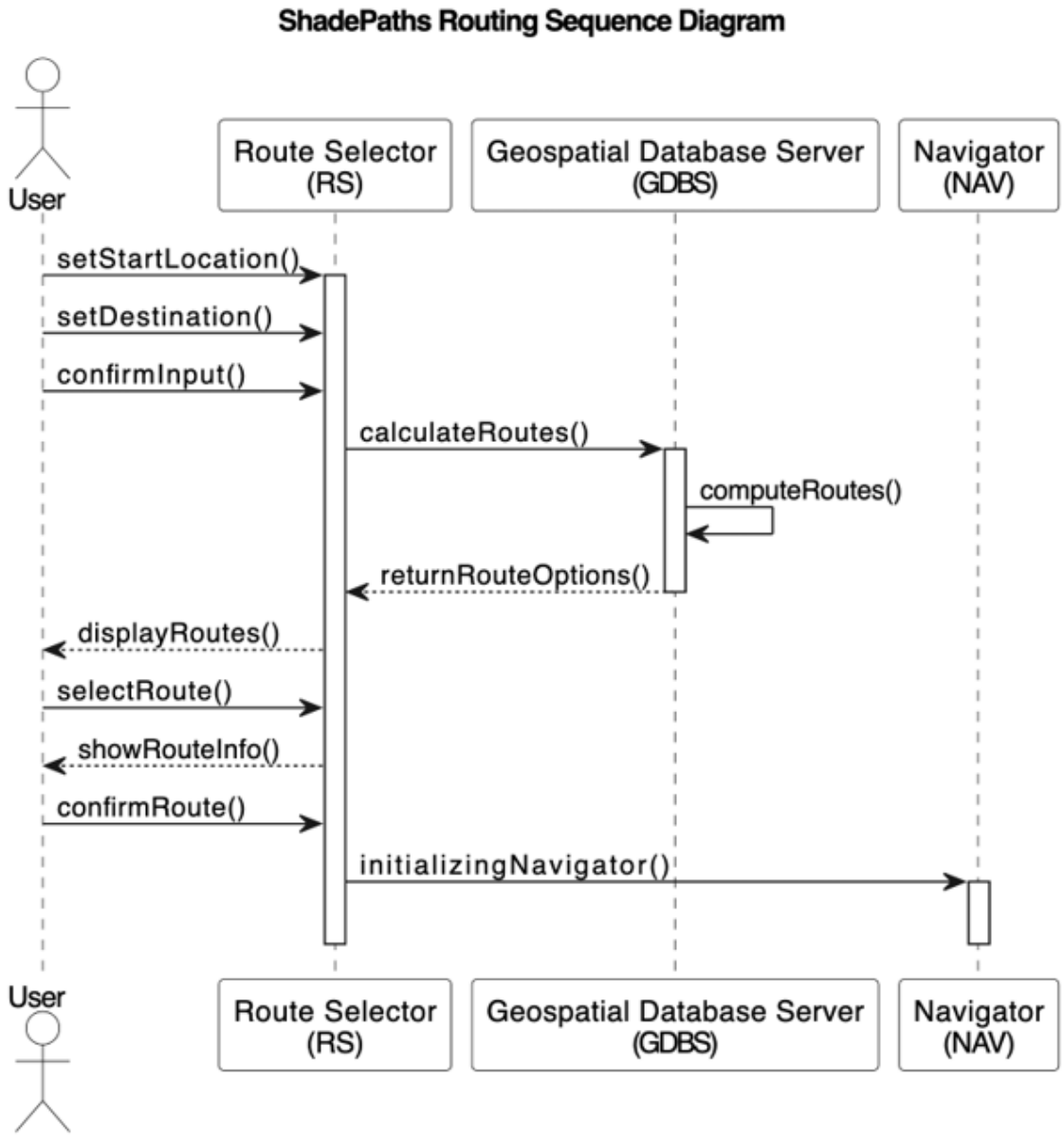

Figure 5: Sequence Diagram

Description

The above diagram shows the path of a route through the ShadePaths system, from receiving route input from the user through to the start of navigation. The purpose of the diagram is to represent the interactions that occur during route generation, route selection, and entry into the navigation stage.

The Route Selector can be viewed as the central hub of the routing process, where the user enters the route, the workflow is managed, and the system interacts with the other subsystems. The actual generation of the route is done by the Geospatial Database Server (GDBS), which computes and returns the route options that are available.

The diagram shows a self call within the GDBS: the computeRoutes() call. This shows that the route calculation actually happens within the GDBS after receiving a route calculation request. This was added because route generation is one of the more vital parts of the system, and shows that it is

happening on the GDBS rather than in the Route Selector.

The diagram also shows the interaction between subsystems transferring work. Once the user has selected and confirmed the chosen route the role is transferred from the Route Selector to the Navigator subsystem, and the diagram can be seen as showing the process of the whole routing stage up to the start of navigation with individual subsystems' roles distinct.

2.6. Activity Diagram

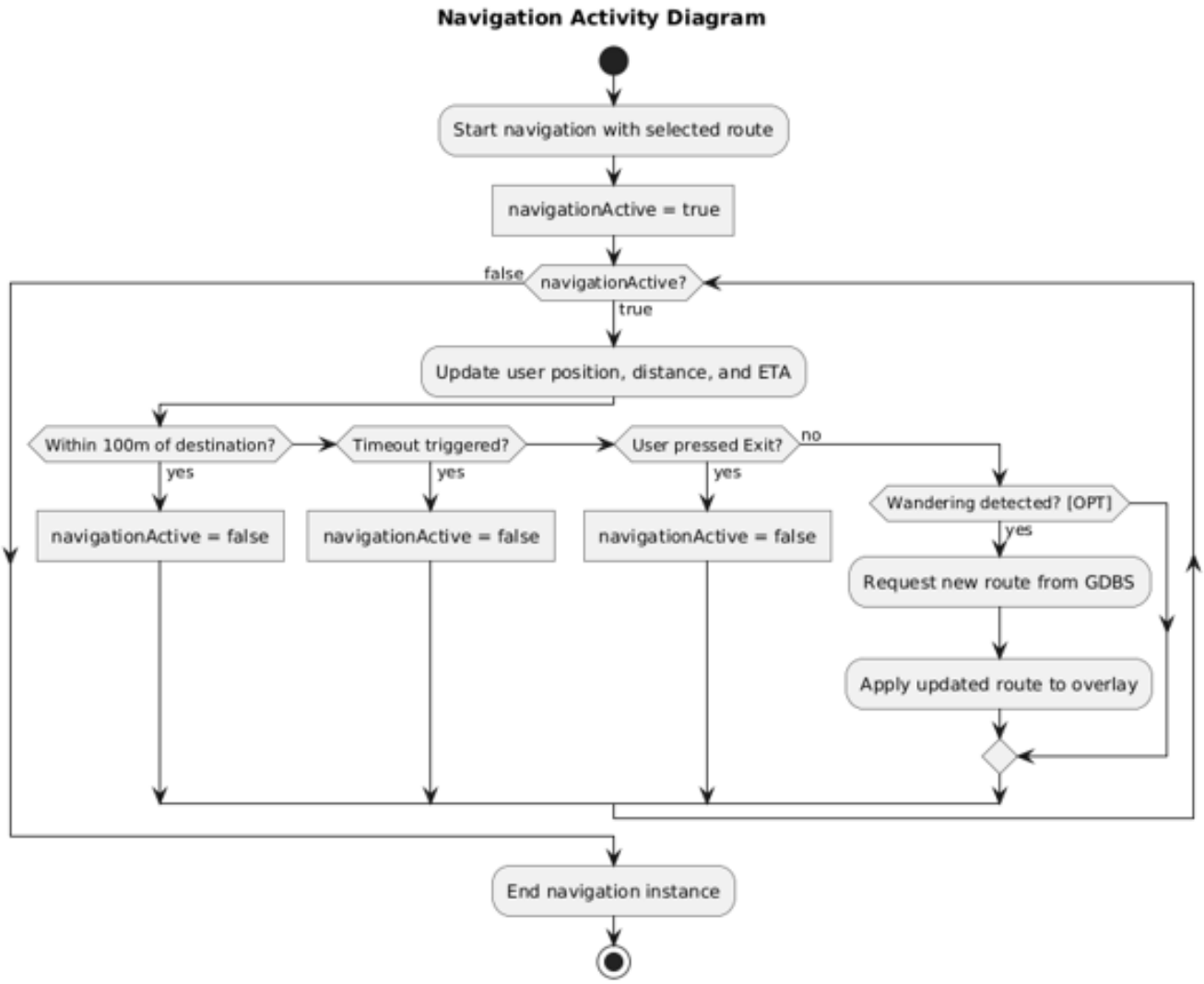

Figure 6: Activity Diagram

Description

The Navigator is implemented as a polling loop, chosen over an event-driven approach to give explicit control over update frequency and exit condition evaluation order. This implementation model is not specified in the SRS (p. 13, Table 10). The three exit conditions are if / else if, not independent listeners. Proximity is evaluated first, so arrival always produces a clean termination even if timeout has also elapsed simultaneously. The position update occurs before exit conditions are checked, so the final ETA and distance reflect the terminal position (SRS p. 13, Table 10). Lateness and routeRelevancyInterval from the class diagram (2.2) are the two values compared at the wandering

decision node (SRS p. 13, Table 10). When wandering triggers, the session continues uninterrupted.

Rationale

Tables 10 and 11in the SRS define the Navigator's stimuli and responses in isolation but provide no execution order or state dependencies needed for implementation. This diagram resolves that by modelling the full runtime polling cycle, making explicit the precedence of exit conditions over positional checks. The three termination guards (proximity, timeout, and user exit) are evaluated sequentially before any optional wandering logic, since session termination takes priority over rerouting. The GDBS interaction appears only within the optional wandering branch, isolating the one server-side call this subsystem triggers. Together, the diagram covers all stimuli in Table 10 and satisfies REQ-NAV-1 through REQ-NAV-3.